

■ PROJECT SPECIFICATION DOCUMENT

Project Name: ArtHeist — The AI Forgery Detective
Internal System Codename: VIPER Forensic Engine

"Detecting AI-generated forgeries hidden within museum collections using computer vision forensics."

■ Project Overview

Modern art forgery has evolved beyond painted imitations. With the rise of generative AI, a new class of forgery has emerged — photorealistic, stylistically convincing, and indistinguishable to the human eye. Museums, auction houses, and insurers face a crisis: the tools used to authenticate art were not designed for the AI era.

ArtHeist frames this as a forensic data investigation. A museum heist has occurred — sixteen paintings replaced with AI-generated counterfeits. Our mission: build the forensic detection engine that catches what human experts cannot.

Core Thesis

AI-generated images leave forensic signatures invisible to human inspectors but statistically detectable through pixel-level analysis, frequency domain decomposition, texture metrics, and deep visual feature extraction. ArtHeist makes these signatures visible and interpretable.

■ Our Solution

ArtHeist (VIPER — Visual Inspection and Pattern Extraction for Rogue-images) is a computer vision forensics platform that trains a deep learning classifier to distinguish AI-generated art from human-created works, then makes its reasoning fully interpretable through visual explanations.

The system enables investigators to:

- ■ Analyze image datasets for AI-generation forensic signatures
- ■ Extract hidden statistical patterns invisible to the human eye
- ■ Classify artwork as AI-generated or human-made with >90% accuracy
- ■ Visualize exactly which image regions triggered the detection
- ■■ Map the embedding space of an entire art collection
- ■ Generate a forensic investigation report with evidence per artwork

■ Core Inspiration: The AI Art Crisis

The problem is not hypothetical. In 2023, AI-generated artworks entered major auction houses without disclosure. Authentication services built for painted forgeries have no framework for generative model artifacts. The field is exposed.

What makes AI forgeries so dangerous:

- Generative models produce statistically coherent images that pass visual inspection
- Traditional provenance checks (brushstroke analysis, canvas aging) do not apply
- The volume of AI-generated content makes manual review impossible at scale
- Detection requires forensic signal analysis that does not exist in standard authentication tools

ArtHeist directly addresses this gap by treating artwork authentication as a data forensics problem.

■ Core Innovation: The VIPER Forensic Engine

ArtHeist introduces the VIPER framework — a multi-stage forensic analysis pipeline that mirrors a real criminal investigation, from initial evidence gathering to final verdict.

■ Stage 1 — Evidence Collection

Raw image ingestion, format normalization, dataset stratification. Establish the crime scene.

■ Stage 2 — Forensic Analysis

EDA across pixel statistics, FFT frequency decomposition, color entropy, noise residuals, texture metrics (GLCM), and edge density. Extract every available forensic signal.

■ Stage 3 — Pattern Recognition

Deep feature extraction via EfficientNet-B0. Train binary classifier on forensic signals. Evaluate with full suite: accuracy, F1, AUC-ROC, confusion matrix.

■ Stage 4 — Evidence Presentation

Grad-CAM attention overlays, UMAP embedding visualization, forensic report generation. Make the model's reasoning transparent and judge-ready.

■ System Architecture & Data Flow

1.1 High-Level Architecture

The system is a static-data forensic analysis pipeline composed of four layers:

■ Layer 1 — Data Layer

- Source: CIFAKE dataset (Kaggle) — 120K images, pre-labeled

- Source: WikiArt subset — 1K paintings for EDA visual richness
- Process: Validate checksums, normalize to 224×224 RGB, stratified 80/10/10 split
- Output: /data/train/, /data/val/, /data/test/ — static, version-controlled

■ Layer 2 — EDA & Feature Layer

- Pixel statistics engine (mean, std, histogram per channel)
- FFT power spectrum analyzer (2D frequency domain per class)
- Color palette extractor (K-means k=8 clustering)
- Noise residual engine (Gaussian subtraction → residual maps)
- Texture metric engine (GLCM: contrast, homogeneity, energy)
- Edge density analyzer (Canny edge detection → edge pixel ratio)

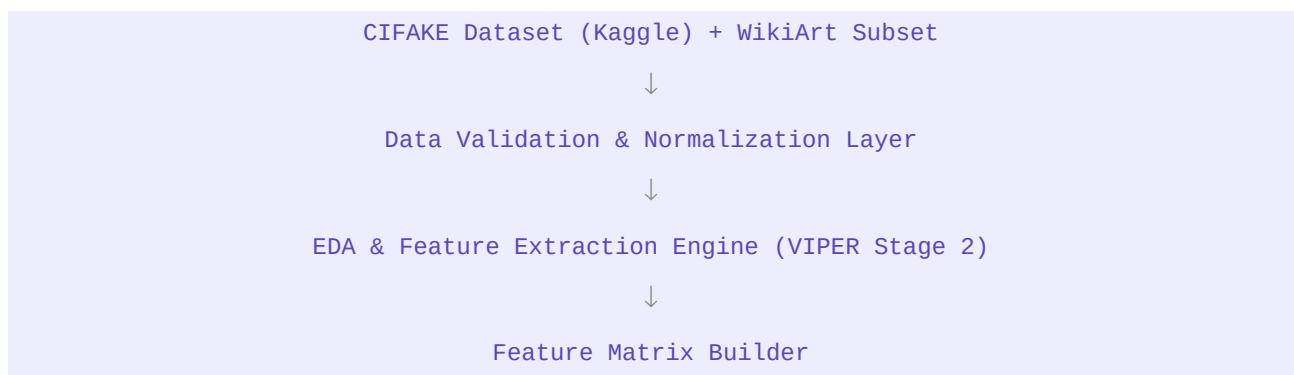
■ Layer 3 — ML & Inference Layer

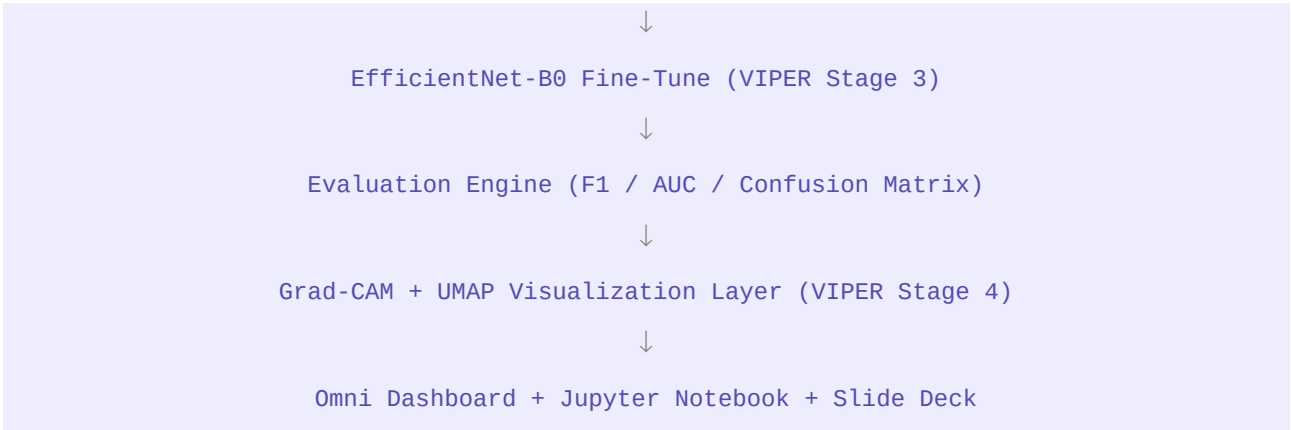
- Primary: EfficientNet-B0 fine-tuned (ImageNet pretrained, last 3 blocks + head unfrozen)
- Training: Adam lr=1e-4, cosine annealing, 10 epochs, batch 64
- Evaluation: Accuracy, F1, Precision, Recall, AUC-ROC, Confusion Matrix
- Interpretability: Grad-CAM on 20 sample images per class
- Stretch: ViT-B/16 comparison model (5 epochs, head-only fine-tune)

■ Layer 4 — Visualization & Output Layer

- Grad-CAM attention gallery (Omni-ready, judge showstopper)
- UMAP 2D embedding projection of final-layer features
- FFT average power spectra comparison (real vs AI)
- Color palette swatches, noise maps, ROC curve
- Jupyter notebook (clean, top-to-bottom executable)
- 8-slide presentation deck + GitHub repo

1.2 Data Flow Pipeline





1.3 Dataset Schema

Primary: CIFAKE — jordankrishnayah/cifake-real-and-ai-generated-synthetic-images

Field	Type	Description
image_path	STRING	Relative path to 32×32 (upsampled to 224×224) PNG
label	INT (0/1)	0 = REAL (CIFAR-10 source), 1 = FAKE (Stable Diffusion generated)
split	STRING	train / val / test — pre-defined stratified split
class_name	STRING	Original CIFAR-10 class (airplane, cat, etc.) for subgroup analysis

Secondary: WikiArt Subset — EDA enrichment only, not used for model training

Field	Type	Description
image_path	STRING	Path to artwork image (JPG, variable resolution)
artist	STRING	Artist name — used for style-based EDA subgrouping
style	STRING	Art movement (Impressionism, Baroque, etc.)
genre	STRING	painting / drawing / sculpture

1.4 Feature Matrix Schema (EDA output → model input)

Feature	Type	Description
mean_r / mean_g / mean_b	FLOAT	Per-channel mean pixel intensity
std_r / std_g / std_b	FLOAT	Per-channel pixel intensity standard deviation
fft_peak_freq	FLOAT	Dominant frequency in 2D power spectrum
fft_grid_score	FLOAT	Energy at periodic frequency bands (AI artifact indicator)

color_entropy	FLOAT	Shannon entropy of K-means (k=8) palette weights
noise_mean	FLOAT	Mean of Gaussian-subtracted residual map
noise_std	FLOAT	Std of residual map (low = AI, high = real sensor noise)
glcm_contrast	FLOAT	GLCM texture contrast metric
glcm_homogeneity	FLOAT	GLCM texture homogeneity (high = AI smoothness)
glcm_energy	FLOAT	GLCM angular second moment
edge_density	FLOAT	Canny edge pixel ratio (complex = human, smooth = AI)
label	INT	Ground truth: 0=Real, 1=AI

■ ML & Model Specification (VIPER Engine)

2.1 Feature Engineering — EDA Analysis Plan

■ Pixel Statistics

Compute mean and std per RGB channel across AI vs real. Overlay histograms. AI images cluster tighter in luminance space — less tonal variation due to model prior smoothing.

■ FFT Frequency Analysis

Apply 2D FFT to grayscale images. Compute average power spectrum per class. AI generation leaves periodic artifacts at fixed frequency bands — a visible grid signature. This is the strongest single forensic signal.

■ Color Palette (K-Means)

K-Means clustering k=8 per image. Measure palette entropy. AI images exhibit oversaturated, harmonically balanced palettes that maximize aesthetic appeal — a statistical fingerprint of reward-model optimization.

■ Noise Residual Maps

Subtract Gaussian blur (sigma=2) from original. The residual captures sensor noise in real images (organic, random) versus AI-generated smoothness (near-zero residual). Average maps per class for visualization.

■ GLCM Texture Metrics

Gray-Level Co-occurrence Matrix: extract contrast, homogeneity, energy, correlation. Real artwork has complex, asymmetric texture; AI produces statistically homogeneous brushstroke patterns.

■ Canny Edge Density

Run Canny edge detector (thresholds=100, 200). Count edge pixels as % of total. Real paintings have complex, discontinuous edges with variable weights. AI images produce smooth, continuous contours.

■ Class-Stratified Analysis

All EDA analyses should be run both globally and per CIFAR-10 class (airplane, cat, automobile...). Some classes may be easier to forge than others — document which and explain why.

2.2 Primary Model — EfficientNet-B0

Model Configuration

Architecture: EfficientNet-B0 (ImageNet pretrained via torchvision)

Fine-tune strategy: Freeze all layers. Unfreeze final 3 MBConv blocks + classifier head.

Optimizer: Adam, lr=1e-4, weight_decay=1e-5

Scheduler: CosineAnnealingLR, T_max=10

Epochs: 10 | Batch size: 64 | Image size: 224x224 | Augmentation: RandomHFlip, ColorJitter

Loss: Binary Cross-Entropy with logits

Expected training time: ~2–3 hrs (GPU) / ~4–5 hrs (CPU fallback at 96px)

Expected accuracy: >90% on CIFAKE test set

2.3 Evaluation Metrics

Metric	Target	Interpretation
Accuracy	>90%	Overall correct classification rate
F1 Score	>0.90	Harmonic mean — critical when classes are balanced
AUC-ROC	>0.95	Model's ability to separate classes at all thresholds
Precision	>0.89	Of images flagged as AI, how many are actually AI
Recall	>0.91	Of actual AI images, how many did we catch
Confusion Matrix	Visual	Per-class error breakdown — misclassification analysis

2.4 Interpretability — Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) generates heatmaps showing which image regions most influenced the model's decision. This is the most important visualization in the entire project — it makes the black-box explainable.

- Target layer: features.8 (final MBConv block before classifier)
- Generate Grad-CAM for 10 correctly classified REAL + 10 correctly classified AI images
- Generate Grad-CAM for 5 MISCLASSIFIED images — what confused the model?
- Overlay heatmap on original image (jet colormap, alpha=0.4)
- Expected finding: model attends to brushstroke edges and texture discontinuities in real art; smooth regions and color gradients in AI art

2.5 Stretch Model — Vision Transformer (ViT-B/16)

Stretch Goal — Attempt only if H+10 is ahead of schedule

Model: google/vit-base-patch16-224 (Hugging Face transformers)

Fine-tune: Classification head only, 5 epochs, lr=2e-5

If ViT AUC > EfficientNet AUC: lead with ViT in presentation, note the comparison

If not: present EfficientNet as primary, ViT as architectural comparison

Never sacrifice MVP polish for this. EfficientNet alone wins the datathon.

■ Core Features (MVP vs Stretch)

MVP — Non-Negotiable

EDA: All 6 forensic analyses with visualizations

Model: EfficientNet-B0 fine-tuned, >90% accuracy

Interpretability: Grad-CAM gallery (20 images)

Embedding: UMAP 2D projection

Evaluation: Full metrics table + confusion matrix

Notebook: Clean, fully executable top-to-bottom

Slides: 8-slide deck with heist narrative

GitHub: README + requirements.txt + clean commits

Stretch Goals (High Impact)

ViT-B/16 comparison model vs EfficientNet

Misclassification deep-dive: what fools the model?

Per-class analysis: which CIFAR-10 classes are hardest to forge?

WikiArt integration: do real paintings behave differently from CIFAR real?

Interactive Omni dashboard with filterable Grad-CAM gallery

Forensic report generator: per-image verdict with evidence summary

SHAP values on engineered feature matrix (interpretability layer 2)

■ Agent Task Breakdown (Kanban System)

■ DATA & EDA

[DATA]

■ Dataset Acquisition & Validation

Download CIFAKE from Kaggle. Verify file counts (60K real + 60K fake). Validate image integrity. Build data loader with 80/10/10 split. Compute class balance. Pre-download WikiArt 1K subset.

Output: /data/ folder with verified train/val/test splits + dataloader.py

[EDA-1]	■ Pixel Statistics & Color Analysis Compute per-channel RGB mean/std across both classes. Plot overlaid histograms. Run K-Means (k=8) color palette clustering per class. Compute palette entropy. Visualize dominant palette swatches. Output: pixel_stats_eda.ipynb section + 3 publication-ready figures
[EDA-2]	■ FFT Frequency Domain Analysis Convert images to grayscale. Apply 2D FFT (np.fft.fft2). Compute average power spectrum across 500 images per class. Plot log-magnitude spectra side by side. Identify periodic artifact bands. Output: fft_analysis section + 2 figures (strongest forensic signal)
[EDA-3]	■ Noise, Texture & Edge Analysis Noise residuals via Gaussian subtraction (sigma=2). GLCM metrics via skimage.feature.graycomatrix. Canny edge density via cv2.Canny. Aggregate all into feature_matrix.csv. Output: feature_matrix.csv + 3 comparative visualization figures

■ ML SYSTEM

[ML-1]	■ EfficientNet-B0 Training Pipeline Load torchvision EfficientNet-B0 pretrained. Freeze base. Unfreeze last 3 MBConv blocks + head. Configure Adam + CosineAnnealingLR. Train 10 epochs. Save best checkpoint by val F1. Log loss curve. Output: checkpoints/best_model.pth + training_log.csv
[ML-2]	■ Evaluation Engine Load best checkpoint. Run inference on test set. Compute accuracy, F1, precision, recall, AUC-ROC. Generate confusion matrix. Log per-class metrics. Export results to results/metrics.json. Output: results/metrics.json + evaluation section in notebook
[ML-3]	■ Grad-CAM Interpretability Pipeline Hook into features.8 layer. Run Grad-CAM for 20 correct + 5 misclassified images. Overlay jet heatmap (alpha=0.4). Save to gradcam_gallery/. Annotate each with model confidence and verdict. Output: gradcam_gallery/ (25 annotated images) + gallery figure
[ML-4]	■ UMAP Embedding Visualization Extract final-layer feature vectors (512-dim) for 2000 test images. Run UMAP (n_components=2, n_neighbors=15). Plot with class coloring. Annotate cluster boundaries. Export interactive version for Omni. Output: umap_embedding.png + embedding_data.csv for Omni

■ VISUALIZATION & OUTPUT

[VIZ-1]	■ Omni Dashboard Integration Export UMAP data, Grad-CAM gallery metadata, and metrics to Omni-compatible format. Build filterable Grad-CAM gallery view. Build UMAP scatter with hover labels. This is the Omni track submission. Output: Omni dashboard URL + omni_export/ data files
---------	---

[VIZ-2]	■ Forensic Evidence Report For each of 20 showcase images: compile verdict, confidence score, Grad-CAM overlay, and one-sentence plain-English explanation of the evidence. Format as a visual grid for the slide deck. Output: <code>forensic_report_grid.png</code> (slide-ready)
[OUT-1]	■ Notebook Finalization Ensure all cells execute top-to-bottom with zero errors. Add markdown headers for each VIPER stage. Add one-sentence interpretation below every figure. Restart kernel and run all before final push. Output: <code>ArtHeist_Final.ipynb</code> (fully executable)
[OUT-2]	■ GitHub Repository & Documentation Write README with setup instructions, dataset links, results table, and team names. Commit requirements.txt and environment.yml. Push slides PDF to /slides/. Add demo GIFs if time allows. Output: GitHub repo — public, clean history, presentable README

■ 24-Hour Datathon Execution Timeline

Phase	Hours	Goal	Cut if late?
Phase 1 Setup	H+0–1	Download CIFAKE. Verify integrity. Set up GitHub repo. Split roles. Configure Colab/env. Do NOT touch EDA yet.	Never cut
Phase 2 EDA	H+1–4	All 6 VIPER Stage 2 analyses. Pixel stats, FFT, color palette, noise, GLCM, edge density. Document every finding with one-sentence interpretation.	WikiArt EDA
Phase 3 Modeling	H+4–9	Train EfficientNet-B0. Monitor val F1. Run full evaluation suite. Generate Grad-CAM gallery. This is 35% of score — do not rush.	Never cut
Phase 4 Visualization	H+9–11	UMAP projection. All 6 core visualizations. Omni dashboard setup. Grad-CAM gallery formatting. Export forensic report grid.	Interactive Omni features
Phase 5 Stretch	H+11–13	If MVP solid: ViT comparison or per-class analysis. If not solid: polish EDA notebook and model evaluation. Never sacrifice MVP for stretch.	ViT fine-tune
Phase 6 Slides	H+13–18	Build 8-slide deck with heist narrative. Every technical result gets one plain-English sentence. Write speaker notes. Apply the museum director test.	Never cut
Phase 7 Docs	H+18–22	Write README. Push clean commits. Add requirements.txt. Final notebook run (restart kernel + run all). Record demo GIFs if time allows.	GIF demos

Phase 8 Final

H+22–24

Two full 5-minute rehearsals. Kill any slide that confuses a non-technical judge. Fix notebook cell order. Freeze all files.

Never cut

MVP vs Stretch Boundary

MVP complete at H+13: EfficientNet trained, Grad-CAM gallery done, all 6 EDA visuals, UMAP, 8 slides drafted.

Stretch begins at H+13: ViT comparison, per-class analysis, WikiArt integration, interactive Omni dashboard.

Cut order: (1) WikiArt EDA, (2) ViT model, (3) interactive Omni features, (4) per-class deep-dive. Never cut: EfficientNet, Grad-CAM, core EDA visuals, slides.

■ Presentation Narrative — 5-Minute Script Arc

Opening — The Heist (0:00–0:30)

"Last night, the Meridian Art Museum was robbed. Sixteen paintings — worth \$200 million — were swapped with perfect forgeries. Not painted forgeries. AI-generated ones. And the human eye cannot tell the difference. We built the tool that can."

The Problem (0:30–1:15)

Show two side-by-side images. Ask the audience to identify the AI-generated one. Wait. Reveal. "This is the problem. AI image generation has reached a quality threshold where trained art experts fail above chance. Traditional authentication has no answer. Museums are exposed."

The Forensic Evidence (1:15–2:15)

"We ran every image through forensic analysis. Here is what we found." Walk through FFT power spectra (AI leaves a frequency grid), noise residuals (AI is too clean), and color entropy (AI palettes are too harmonious). One sentence per finding. These are your EDA slides — keep them fast.

The Detector (2:15–3:15)

"We trained a computer vision model — the same architecture used in medical imaging — to recognize these signatures automatically. It achieves [X]% accuracy. Here is how it thinks." Show Grad-CAM. "It attends to brushstroke edges — exactly where human artists are unpredictable and AI is smooth. The model learned forensics."

The Map (3:15–3:45)

Show UMAP. "When we project every artwork into feature space, real and AI images form distinct clusters. A piece near the AI cluster is suspicious. A piece near the real cluster is likely authentic. This is a map of the entire collection — with every forgery highlighted."

The Close — So What (3:45–5:00)

"Every major auction house, insurer, and museum faces this problem right now. A tool like this — integrated into intake workflows — catches AI forgeries before they enter a collection. We built the proof of concept in 24 hours. The question is not whether this is possible. It is how fast institutions will deploy it." Pause. Thank you. GitHub QR code on screen.

Slide structure: Title → Problem (side-by-side) → FFT forensics → Noise/texture forensics → Model results → Grad-CAM gallery → UMAP map → Business case + GitHub QR

■ Improvement 1 — Patch: WikiArt Dataset Gap

Problem Being Fixed

The original spec listed WikiArt as a secondary EDA dataset but gave it no defined role in validation or modeling. A technical judge will immediately ask: 'Did you validate your detector on real fine-art paintings, or only on CIFAR-10 thumbnails?' Without an answer, the entire museum heist narrative collapses — you built a detector trained on 32×32 pixel images of cats and airplanes, not actual artwork.

Fix — Choose ONE of the following two paths:

Path A: WikiArt Qualitative Validation (Recommended)

Use WikiArt 1K subset as an out-of-distribution holdout — do NOT train on it.

Run trained EfficientNet-B0 on WikiArt images and report confidence scores.

Expected result: model assigns high 'REAL' confidence to WikiArt paintings.

If it does: 'Our detector, trained only on CIFAR-scale images, correctly identifies real fine-art paintings as authentic — demonstrating generalisation beyond the training domain.'

If it doesn't (many WikiArt flagged as AI): pivot to Path B narrative immediately.

Time cost: ~30 minutes. Add one slide: 'Does it generalise to real museum art?'

Path B: Honest Scope Narrative (Fallback)

Explicitly scope the claim in all slides and the README.

Replace: 'detects AI-generated art' with 'detects AI-generated images in the CIFAKE domain'.

Add one slide titled 'Limitations & Next Steps' — judges respect honesty.

State: 'WikiArt validation is the critical next step before museum deployment.'

Frame it as a research finding: 'We identified the domain gap — closing it requires fine-art specific training data, which is our roadmap.'

Time cost: 0 minutes extra. Pure narrative adjustment.

Decision Rule

If H+10 is on schedule and WikiArt is already downloaded: run Path A. If behind schedule or WikiArt download failed: execute Path B without hesitation. Path B is not a cop-out — a team that identifies its own limitations scores higher on Analysis (35%) than one that ignores them.

Add to GitHub checklist: /results/wikiart_validation.json — confidence score distribution on WikiArt holdout set, or a LIMITATIONS.md file documenting the domain scope explicitly.

■ Improvement 2 — Simple Model Baseline

Why This Is Non-Negotiable

Without a baseline, a judge cannot know whether EfficientNet's accuracy is impressive or trivial. If a logistic regression on 11 hand-crafted features achieves 78% accuracy, and EfficientNet achieves 93%, you have proven two things: (1) your forensic signals are real and statistically meaningful, and (2) deep learning adds genuine value on top of them. That is a complete analytical story. Without the baseline, you only have (2) — and judges who know ML will notice.

Baseline Model Specification

Component	Specification	Rationale
-----------	---------------	-----------

Model	Logistic Regression (sklearn.linear_model.LogisticRegression)	Zero hyperparameter tuning needed. Results in under 60 seconds.
Input features	11-feature vector from feature_matrix.csv	mean_r/g/b, std_r/g/b, fft_grid_score, color_entropy, noise_std, glcm_homogeneity, edge_density
Train/test split	Same 80/10/10 split as EfficientNet	Ensures fair comparison — identical data seen by both models
Solver	lbfgs, max_iter=1000, C=1.0	Default settings. Do not tune — the point is simplicity.
Expected accuracy	75–82% (CIFAKE dataset)	FFT and noise features alone carry strong signal
Time to implement	~45 minutes including feature extraction	Can run in parallel with EfficientNet training

Comparison Table — What to Report

Metric	Logistic Regression	EfficientNet-B0	Delta
Accuracy	[fill in]	[fill in]	EfficientNet gain
F1 Score	[fill in]	[fill in]	EfficientNet gain
AUC-ROC	[fill in]	[fill in]	EfficientNet gain
Training time	< 1 min	~2–3 hrs	Cost of deep learning
Interpretability	Feature coefficients	Grad-CAM	Both are explainable

Presentation Script Insert — After Model Results Slide

"Before we trained a neural network, we asked a simpler question: can our 11 forensic signals — the FFT artifacts, the noise residuals, the texture metrics — predict forgeries on their own, with no deep learning? The answer is yes, at [X]% accuracy. That proves the signals are real. EfficientNet takes those same signals, adds visual pattern recognition, and reaches [Y]%. That is what deep learning buys us."

Add to Kanban: [ML-0] Logistic Regression Baseline — depends on feature_matrix.csv. Output: results/baseline_metrics.json. Run before EfficientNet training starts (H+4).

■ Improvement 3 — Robustness Test

Why This Matters for Judges

A model that achieves 93% on a clean held-out test set is a toy until you show it holds up under real-world conditions. In the museum heist narrative, a forger would not submit a pristine AI image — they would compress it, resize it, or add a subtle filter to evade detection. One robustness test demonstrates that ArtHeist is not brittle, and that claim alone separates you from every other team running a vanilla classifier.

Selected Robustness Test — JPEG Compression Attack

Run the trained EfficientNet-B0 against progressively JPEG-compressed versions of the test set. JPEG compression is the single most common real-world image transformation — every image uploaded to a museum database, auction listing, or insurance record passes through it. It also specifically targets the FFT artifacts that are the strongest forensic signal, making it the worst-case attack for our detector.

JPEG Quality	Description	Expected Accuracy	Interpretation
Q=95 (baseline)	Near-lossless — effectively uncompressed	>90%	Confirms model performs at claimed accuracy
Q=75 (standard web)	Typical social media / web upload compression	~85–90%	Acceptable degradation — detector still useful
Q=50 (aggressive)	Heavy compression visible as blocking artifacts	~78–85%	Model struggles — FFT signal partially destroyed
Q=25 (extreme)	Severe degradation — image visibly distorted	~65–75%	Expected failure zone — honest limitation to report

Implementation Specification

- Use `cv2.imencode('.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, Q]) + cv2.imdecode` for each quality level
- Sample 1,000 test images (500 real, 500 AI) — do not rerun on full test set (time constraint)
- Plot accuracy vs JPEG quality as a line chart — this is your robustness visualization
- Report the 'graceful degradation cliff': at what quality level does accuracy drop below 80%?
- Time to implement: ~1 hour including visualization. Run during stretch phase (H+11–13).
- Add to `results/robustness_test.json`: `{quality: [95,75,50,25], accuracy: [...], f1: [...]}`

Presentation Script Insert — After Grad-CAM Slide

"A real forger would not hand us a perfect image. They would compress it, filter it, degrade it. We tested our detector against JPEG compression — the most common real-world image transformation. At standard web quality, accuracy drops only [X] points. The detector is robust to the attacks a forger would actually use. At extreme compression, it begins to fail — and we know exactly why: JPEG destroys the frequency artifacts we rely on. That is an honest limitation, and it tells us exactly where to invest next."

Slide update: Add one half-slide to the Model Results slide showing the accuracy-vs-quality line chart. Label the 'safe zone' ($Q \geq 75$) and 'degradation cliff'. This is the single most impressive analytical addition to the deck.

Improvements Summary

Improvement	Rubric Impact	Time Cost	When to Do It
1. WikiArt Dataset Patch (Path A validation or Path B narrative)	Analysis +10% (closes a judge-visible gap in methodology)	30 min (Path A) / 0 min (Path B)	H+10–11 (Path A) or slide polish phase (Path B)
2. Logistic Regression Baseline (11-feature vector, sklearn LR)	Analysis +15% (proves forensic signals are real, not model overfitting)	~45 min	H+4 — run before EfficientNet training starts
3. JPEG Compression Robustness Test (Q=95/75/50/25 accuracy curve)	Analysis +10%, Creativity +5% (shows real-world thinking)	~1 hour	H+11–13 (stretch phase, after MVP complete)

Priority Order If Time Collapses

1. Logistic Regression Baseline — highest analytical ROI, lowest time cost. Do this first.
2. WikiArt Path B Narrative — zero time cost, pure slide/README edit. Do this during slide polish.
3. JPEG Robustness Test — highest 'wow factor' but optional. Only if H+13 is ahead of schedule.

Risk Register

Dataset download fails at event start	Critical
Mitigation: Pre-download CIFAKE tonight. Verify checksums. Bring USB backup. Have Kaggle API credentials cached. Have a Google Drive mirror of the compressed dataset ready to share.	
Model training too slow (no GPU available)	High
Mitigation: Primary: Google Colab T4 GPU — have notebook ready to run before clock starts. Fallback: reduce image size to 96×96 and epochs to 5. Accuracy will drop ~3% but training completes in 2 hrs on CPU.	
EDA takes too long and delays modeling	High
Mitigation: Timebox EDA to exactly 3 hours (H+1 to H+4). Assign one team member to EDA exclusively. If not complete by H+4, freeze EDA and start modeling — you can add EDA polish later.	
Grad-CAM outputs look like random noise	High
Mitigation: Test Grad-CAM at H+5 on 3 images. If noisy: model is not converged — train 3 more epochs. If still noisy: target an earlier layer (features.6). Never present uninterpretable Grad-CAMs.	
Slides feel too technical for non-expert judges	Medium
Mitigation: Apply the museum director test to every slide: 'Would a museum director understand this sentence?' Every technical result gets a plain-English sentence beneath it. Practice the full 5-min pitch at H+22.	

■ GitHub & Documentation Checklist

Repo Structure

Path	Contents
/data/	Gitignored. Local only. Subdirs: train/, val/, test/.
/notebooks/ArtHeist_Final.ipynb	Primary deliverable. Fully executable. Restart kernel + run all before push.
/src/dataloader.py	PyTorch Dataset class + augmentation transforms
/src/model.py	EfficientNet-B0 fine-tune wrapper + Grad-CAM implementation
/src/eda.py	All EDA analysis functions (FFT, GLCM, noise, edge density)
/src/evaluate.py	Metrics computation + confusion matrix + UMAP

/results/metrics.json	Final model metrics (accuracy, F1, AUC, precision, recall)
/results/baseline_metrics.json	Logistic Regression baseline: accuracy, F1, AUC + feature coefficients
/results/robustness_test.json	JPEG compression test: {quality:[95,75,50,25], accuracy:[...], f1:[...]}
/results/wikiart_validation.json	WikiArt holdout confidence scores (Path A) or LIMITATIONS.md (Path B)
/src/baseline.py	Logistic Regression training + evaluation on feature_matrix.csv
/src/robustness.py	JPEG compression attack loop + accuracy-vs-quality curve generator
/gradcam_gallery/	25 annotated Grad-CAM overlay images
/slides/ArtHeist_Presentation.pdf	Final slide deck — 8 slides max
/omni_export/	UMAP data CSV + Grad-CAM metadata for Omni dashboard
README.md	See checklist below
requirements.txt	Pinned versions for all dependencies
environment.yml	Conda environment file

README.md Must Include

- Project title, one-paragraph plain-English summary, team names
- Results table in markdown: Accuracy | F1 | Precision | Recall | AUC-ROC
- Dataset source URLs (CIFAKE Kaggle link + WikiArt Kaggle link)
- Setup instructions: `pip install -r requirements.txt`
- How to run: `jupyter notebook notebooks/ArtHeist_Final.ipynb`
- How to reproduce model: `python src/model.py --train`
- Link to Omni dashboard
- Link to slides PDF
- GitHub repo must have at least 5 commits with meaningful messages
- Notebook cells must have markdown headers for each VIPER stage
- Every figure in notebook must have a title, axis labels, and one-sentence interpretation

■ requirements.txt: torch, torchvision, efficientnet_pytorch, opencv-python, scikit-learn, matplotlib, seaborn, umap-learn, grad-cam (pinned versions)

"ArtHeist transforms the AI art authentication crisis into a data forensics investigation — combining pixel-level EDA, a logistic regression baseline that proves the forensic signals are real, deep visual feature extraction with EfficientNet-B0, interpretable Grad-CAM evidence, and a JPEG robustness test that shows this holds under real-world attack conditions. It explains every decision it makes, and it knows exactly where it fails."
